DRISHTI · दृष्टि · SIGHT

Drishti VaR engine
Architecture, data contract and how VaR works

Any set of trades or positions, in any reporting currency

Ashutosh Sinha

October 2026

Release 2.9.4 · historical simulation over 509 scenario days, window and confidence configurable

IN THIS DECK

1 Why, and how VaR works
2 Architecture, hierarchies, inputs
3 PVs, intraday, contracts, the UI
4 Scale, sizing, capacity, hardware
5 Storage, deployment, operations
6 Controls, roadmap, rollout

Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.

WHY IT MATTERS

VaR at a large bank: scale, sources, currencies, intraday

≈ 2.3M
positions valued every business date

≈ 15,000
portfolios, each a VaR on demand

5
reporting currencies built ahead: USD, EUR, CAD, JPY, GBP

≈ 700
limit letters: a hierarchy configured by columns

Illustrative volumes of a large bank (an example, not a client's figures); every other figure in this deck is Drishti's.

Scale

- A node's VaR from its float64 aggregate in under a millisecond, whatever its size
- Position grain keeps rows = positions; portfolios are sums of positions

Pricing systems keep their formats

- Scenario PVs or P&L, by trade or by position, files or Kafka
- Each system its own form; systems mix on one date and may switch day to day

Multi-currency reporting

- PVs in each trade's or position's own currency, converted with each scenario's FX
- A EUR or CAD VaR is not the USD VaR × today's rate: it carries the currency risk

Intraday

- Only the changed trades or positions, applied as new − old up every tree
- The official end-of-day record stays the full run's; intraday never restates it

Status words

BUILT in the code and tested
2.9.5 ships in 2.9.5
2.9.6 ships in 2.9.6
DESIGNED cluster scale-out only
PROPOSED this deck's recommendation
MEASURED on a workstation
ESTIMATED scaled from measured
ASSUMPTION for the bank to confirm

Executive summary

What is it?	A VaR engine in the Drishti server: historical simulation over 509 scenario days (or a set window), VaR 99% (95% by setting) and ES 97.5%, 1 day, on demand for any set of trades or positions: the bank, a legal entity, desk, book, a portfolio, a rule or a basket.
Every input	**One engine, every input: scenario PVs or path P&L, by trade or by position, any currency, files or Kafka. Source systems keep their own formats.**
How does it work?	It never reprices. The pricing systems send scenario PVs (in each trade's or position's own currency) or path P&L, at trade or position level, through files or Kafka. Drishti turns each into one internal form, a P&L vector per trade or position, converting PVs with scenario FX into any reporting currency. A scope's vector is the sum of its trades' or positions' vectors, and VaR, ES and every contribution are read from that one vector.
Can I trust it?	Every answer names its inputs' versions, batch ids, scenario set, method and memory tier; components add up to the VaR exactly; every node gets an official end-of-day record, append-only, kept 7 years.
Does it scale?	Measured: 1 million rows summed in 58 ms (heap tier); 100,000 × 5 dates on the tiered store, a node in 0.48 ms. Sized for a large bank (e.g. 2.3M positions, 15,000 portfolios) on the sizing and capacity slides; sharding is designed.

How VaR works in Drishti: add vectors, then read the tail

1 A P&L vector per trade or position
509 numbers: its P&L if the market moved as on each scenario day

→

2 Add the scope's vectors
a set of trades or positions; overlaps once; longs and shorts net

→

3 Sort the 509 P&Ls
worst first; equal P&Ls: the older scenario first

→

4 Read the tail
VaR: minus the 6th worst; ES: minus the mean of the 13 worst

Worked example

ten scenarios, α = 80% so k = 2; one book holding P1 (T1 long, T2 short) and P2 (T3)

Vector	0	1	2	3	4	5	6	7	8	9
T1	5	−8	3	−12	6	−2	4	−5	7	1
T2	−2	3	−1	4	−3	1	−2	2	−3	0
P2 = T3	−4	2	−6	1	−9	3	−1	0	2	−7
P1 = T1 + T2	3	−5	2	−8	3	−1	2	−3	4	1
Book = P1 + P2	−1	−3	−4	**−7**	**−6**	2	1	−3	6	**−6**

VaR
6
2nd worst of the book: −6

ES
6.5
mean of −7 and −6

Component VaR
1.26 + 4.74
P1 + P2 = 6: Euler, adds up

Diversification
6
standalone 5 + 7 − 6

Incremental, P1
−1
a hedge: removing it raises VaR

The measures at N = 509

Measure	Definition
VaR 99%	minus the 6th worst P&L
ES 97.5%	minus the mean of the 13 worst
Component VaR	Euler on ranks 4–8; adds up to VaR
Component ES	mean over the 13 worst; adds up to ES
Incremental	VaR(scope) − VaR(scope − part)
Marginal	component ÷ |size| × 1,000,000
Diversification	Σ standalone VaR − VaR
Limits	per node, portfolio; amber 75%, red 90%

Kupiec test (Paul Kupiec, 1995): is the number of VaR breaches (losses bigger than VaR) consistent with 99%, about 2–3 in 250 days?
Christoffersen test (Peter Christoffersen, 1998): do breaches cluster together, a sign the model reacts too slowly when markets turn?
Basel traffic light: breaches in 250 days at 99%: 0–4 green, 5–9 amber, 10+ red.

CONFIGURABLE · 2.9.4

The window, the confidence, the currencies and the names are settings

Scenario window

`drishti.var.scenario-window: all | N`

The newest N scenarios of the set (all: 509). Per request, window=250 shows a 1-year view beside the 2-year one: labelled on every answer, never official.

Window	VaR 99%	ES 97.5%
all (509)	6th worst	mean of 13
250	3rd worst	mean of 7

Confidence

`DRISHTI_VAR_CONFIDENCE · DRISHTI_VAR_ES_CONFIDENCE`

VaR 99% by default, 95% by configuration; ES 97.5% by default; both also per request. Every label follows the answer (VaR 95% · 1 day). Backtesting stays at 99%.

VaR at	509 days	250 days
99% (default)	6th worst	3rd worst
95%	26th worst	13th worst

Choosing the window banks typically use 1 year (≈ 250 days, the regulatory minimum) or 2 years (≈ 500). A shorter window reacts faster to new volatility; a longer one is steadier and keeps more stress days. Compare both with window= before changing the setting.

Currencies, built ahead

`drishti.var.currencies`

USD, EUR, CAD, JPY, GBP, built in the background after the USD book

`currency-retention-dates`

5: the newest dates kept; older ones built again on first ask

Names

`DRISHTI_VAR_ROOT_NAME`

the top of every VaR tree (root-names: per hierarchy)

`config/demo/names.yaml`

the demo's bank and legal-entity names

Inputs and intraday

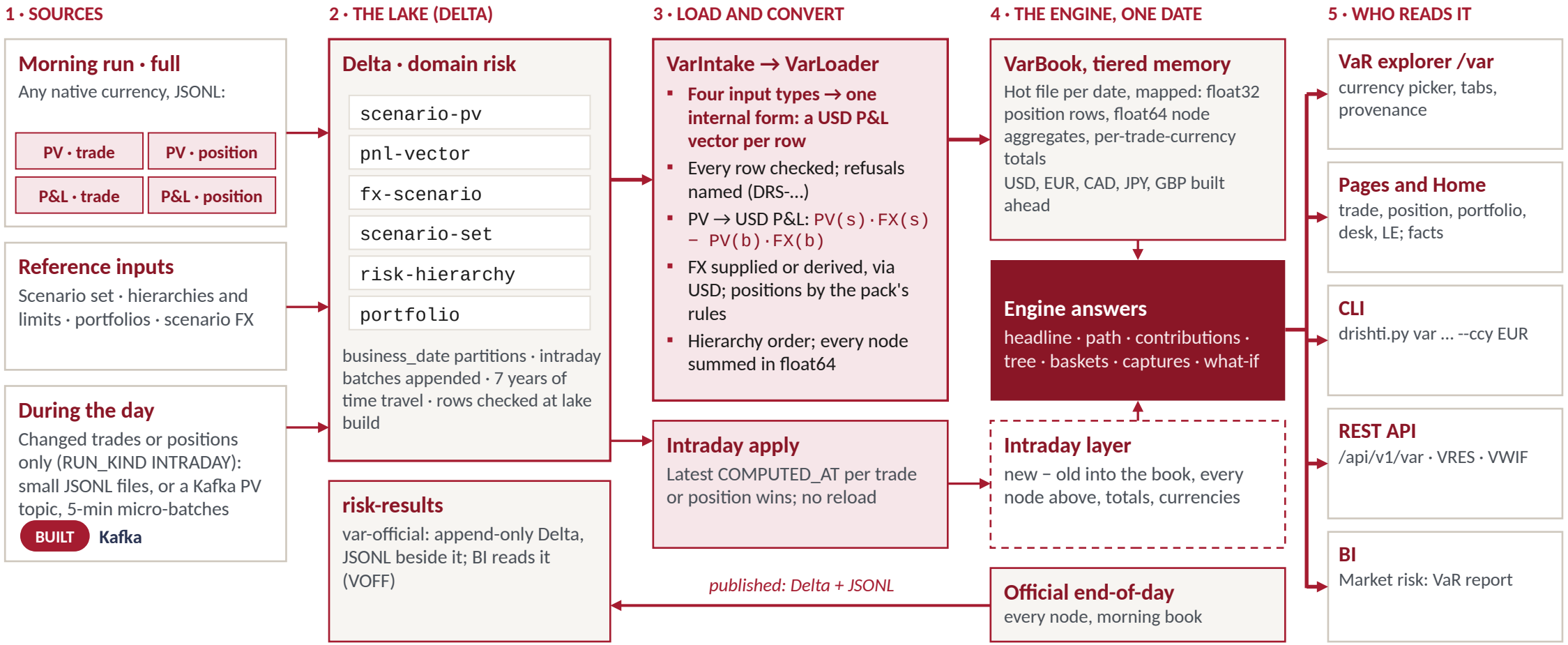
`drishti.var.inputs`

optional pin per source system: form and grain

`intraday.kafka.window`

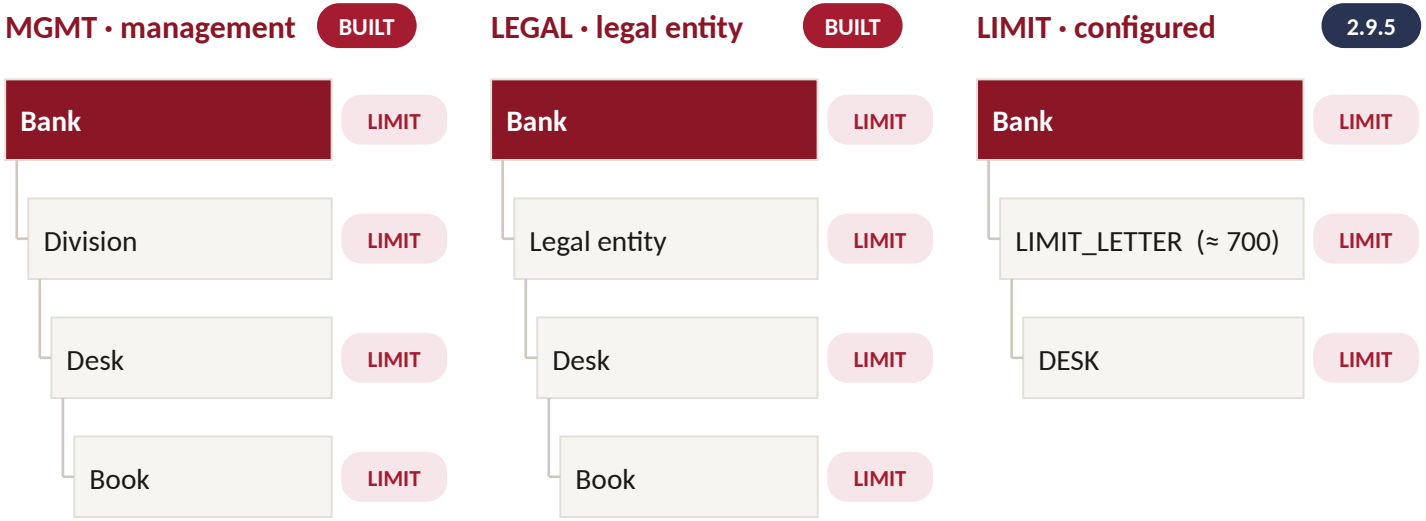
5m; early flush at max-trades (500)

From the pricing systems to every screen, in one direction



Each batch's data-load signal `POST /api/v1/packs/market-risk/loads` *tells the engine: a full run reloads the date; an intraday batch is applied on top. The lake is the record: a restart rebuilds everything, the intraday layer included.*

Three trees: management, legal entity and a configured one



Hierarchies configured by columns 2.9.5

- An ordered list of columns every input row carries: LEGAL_ENTITY > DESK > TT_BOOK, LIMIT_LETTER > DESK, REGION > PRODUCT_TYPE
- Nodes derived from the data; limits attach to node paths from a limits table
- Membership per row: a letter holding part of a book works
- A configured column missing from a source's rows is refused (a DRS code)

Where the limits show

- The tree: each node's VaR, ES and limit bar (amber 75%, red 90%)
- Limit use = VaR ÷ limit; official records carry LIMIT and LIMIT_USE

```
# hierarchies configured by columns · illustrative, ships in 2.9.5
drishti.var.hierarchies:
  LIMIT:  [LIMIT_LETTER, DESK]
  REGION: [REGION, PRODUCT_TYPE]
```

Requires LIMIT_LETTER (or the chosen columns) on every PV, P&L or trade row. MGMT and LEGAL are built today as risk-hierarchy rows; any further tree is configured by columns.

ANY INPUT CONTRACT

One engine, every input: source systems keep their own formats

	Scenario PVs native currency	Path P&L vectors any currency
By trade	✓ BUILT 2.9.4	✓ BUILT since 2.8.0
By position	✓ BUILT 2.9.4	✓ BUILT 2.9.4

Delivery routes

Files, JSONL — BUILT
the morning run; small intraday batches appended

Kafka, micro-batched — BUILT
JSONL messages; 5-min window, early flush, Apply now

Intraday, either route — BUILT
changed trades or positions, as new − old; latest wins

```
{"VALUE_TYPE":"PV",  "GRAIN":"TRADE",    "TRADE_ID":"1000279", "CURRENCY":"EUR", "BASE_PV":…, "PVS":[…509…], "COMPUTED_AT":"…", …}
{"VALUE_TYPE":"PV",  "GRAIN":"POSITION", "POSITION_ID":"POS-…", "CURRENCY":"JPY", "BOOK":"…", "BASE_PV":…, "PVS":[…], …}
{"VALUE_TYPE":"PNL", "GRAIN":"TRADE",    "TRADE_ID":"MX-…",    "CURRENCY":"USD", "PNL":[…509…], …}
{"VALUE_TYPE":"PNL", "GRAIN":"POSITION", "POSITION_ID":"POS-…", "CURRENCY":"USD", "PNL":[…], …}
```

Mix and switch

- Each system sends its own form and grain; systems mix on one date.
- A system may switch day to day; each date loads as it stands.
- Old files: `drishti.py convert --set …`

```
# optional cross-check
drishti.var.inputs:
  ENDUR:   {form: pv,  grain: trade}
  CALYPSO: {form: pnl, grain: trade}
  default: {form: any, grain: any}
```

Guarantees

- One form and grain per system per date (DRS-5210): no double counting
- Fields missing or wrong (DRS-5208), or against the pin (DRS-5209): refused, never guessed
- Provenance shows what each system sent

Caveat a P&L vector sent without its PV converts to another reporting currency only approximately (at each scenario's rate, without the FX move of its base value; counted as `tradesWithoutPv`). Scenario PVs give exact multi-currency VaR.

What a pricing system sends

Every record JSONL: one JSON object a line, UTF-8, .jsonl or .jsonl.gz; files or Kafka

Field	Values	Rule
`VALUE_TYPE`	PV or PNL	required
`GRAIN`	TRADE or POSITION, with its id	required
`CURRENCY`	ISO code; PVs in their own	required
`SOURCE_SYSTEM`	the pricing system that sent it	required
`SCENARIO_SET_ID`	the date's set (demo: HS-509)	all, in order
`BATCH_ID · RUN_KIND`	FULL or INTRADAY	one full run a date
`COMPUTED_AT`	UTC instant	the latest wins
`LEGAL_ENTITY · DESK · TT_BOOK`	where it sits	needed to load
`BASE_PV · SCENARIO_PV`	the PV form	failed: null
`PNL (+ PV)`	the P&L form, USD	PV makes FX exact
`TRADE_STATUS · MATURITY_DATE`	live trades count	CANCELLED: cancel

Files

BUILT

the full run, and small intraday batches; lake build, then the load signal

Kafka

BUILT

JSONL text per message, one or more records; one lake batch per 5-minute window

Intraday and cancels

- RUN_KIND INTRADAY, its own BATCH_ID, a later COMPUTED_AT, the morning's scenario set
- A cancel: TRADE_STATUS CANCELLED, the vector null
- Same form and grain as the system's full run

```
# optional per-system pin (read at start)
drishti.var.inputs:
  SYS_A:   {form: pv,  grain: position}
  SYS_B:   {form: pnl, grain: trade}
  default: {form: any, grain: any}
```

A null scenario PV refuses the whole trade (DRS-5206), never zero-filled. Field names are the as-built contracts'; older files are brought to the record with drishti.py convert --set ….

A checklist for each pricing system

1. **Choose form and grain**
   PVs or P&L, by trade or by position: one form and one grain per system per date
2. **Map the record's fields**
   VALUE_TYPE, GRAIN + id, CURRENCY, SOURCE_SYSTEM; LEGAL_ENTITY, DESK, TT_BOOK; status, maturity
3. **Price over the date's scenario set**
   every scenario in the set's order; a failed one null, never zero
4. **Scenario FX for its currencies**
   send fx-scenario, or let Drishti derive it from the FX history
5. **Pick the route**
   JSONL files for the full run; files or a Kafka topic for intraday
6. **Set the optional pin**
   drishti.var.inputs: a record that disagrees is refused (DRS-5209)
7. **Recompute independently**
   from the sent files, compare bank and desk VaR and ES with the engine
8. **Clear the refusals**
   provenance lists every refused row by code; aim for none before go-live

PROPOSED

Ready when the system's records load with no refusals, the independent recomputation agrees to the cent, and How this number was built shows the system with its form and grain.

From scenario PVs in their own currency to VaR in any currency

Inputs

- Pricing systems send each trade's or position's 509 scenario PVs and base PV in its own currency (`scenario-pv`). The USD P&L-vector form stays supported; a date may mix both, each row in one form.
- Scenario FX: the batch's own (`fx-scenario`) when sent, else derived from the FX history. Every cross goes via USD. Horizon: 1 day.
- Converted when the date loads; positions, nodes, captures and official records do not know which form a row came in.

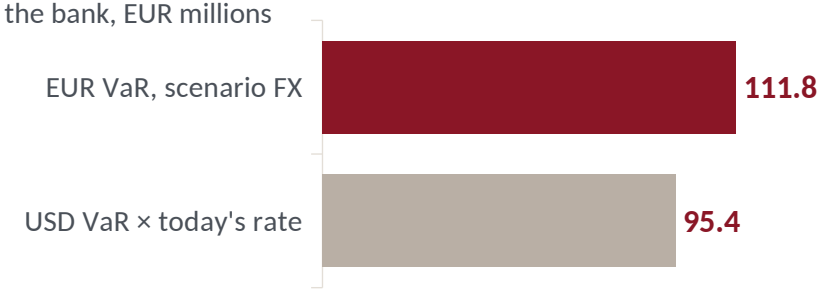
```
USD P&L      PV(s)·FX_c(s) − PV(base)·FX_c(base)
FX derived   FX(s) = FX(base) · H(d_s) / H(d_s − 1)
In r         (p + P&L_USD(s)) / FX_r(s) − p / FX_r(base)
Node in X    Σ_c [PVsum_c(s)·FX_c→X(s) − PVbase_c·FX_c→X(base)]
```

Reporting currencies

- Built ahead in the background: USD · EUR · CAD · JPY · GBP (`drishti.var.currencies`); the newest 5 dates kept; any other currency with FX on first ask.
- Every node keeps its PVs summed per trade currency: a headline in any currency is exact to the cent.
- Trade pages say the Trade currency (EUR · as priced); official records in USD, others per setting.

Why EUR VaR ≠ USD VaR × today's rate

the bank, EUR millions



The bank on 2026-09-30 holds a net 3.65 bn USD of value; on the scenario days it moves against the euro, a risk a euro-based owner carries and a dollar-based one does not.

Ccy	Bank VaR 99%	Bank ES 97.5%
USD	112,041,646	160,143,809
EUR	111,837,045	137,631,546
CAD	189,801,404	230,680,413
JPY	20,883,226,318	23,578,418,999
GBP	104,139,554	125,595,958

On the demo: the 1,339 Endur trades come as scenario PVs (1,089 USD, 246 EUR, 4 CAD). Bank VaR 112,035,906 → 112,041,646 and ES 160,132,107 → 160,143,809: the FX move of the 250 EUR and CAD trades. A USD trade reproduces its vector exactly.

Intraday: only what changed, applied as new – old up the tree

Morning full run
RUN_KIND FULL: every live trade or position; the date loads; official records taken from it

→

Changed rows
Trades or positions amended, new, cancelled; same scenario set; own BATCH_ID, later COMPUTED_AT

BUILT JSONL files

→

Lake: appended batch
Files: drishti.py var intraday --load (lake build --mode append). Kafka: one batch per window

BUILT Kafka micro-batches

→

Apply: new – old
Into the book, every node above (MGMT and LEGAL), the currency totals and built currencies

→

Answers say intraday
Badge in the status line, intraday on every answer; morning hot file never changed

Case	What the engine does
Same trade or position, several batches	latest COMPUTED_AT wins across files and Kafka; an equal one keeps the first
COMPUTED_AT before the morning run	ignored and counted
Cancel (CANCELLED, vector null)	removed from every scope
New trade	book from TT_BOOK, position by the position rules
Moved to another book or position	old ordinal cancelled, added in its new place
Another scenario set	row refused (DRS-5203); the date stays as it is
A full run of the date	date reloads; the intraday layer is dropped

Cost of a batch ∝ rows changed × nodes above × currencies built, never the size of the book.

Kafka, as built in 2.9.4

- A message is JSONL text: one or more records, PVs or P&L, a cancel as a marked line
- A window every 5 min (drishti.var.intraday.kafka.window); early flush at 500 trades; admin “Apply now”
- Each window written to the lake as ONE intraday batch, then applied like a file
- Replaying the topic after a restart writes nothing twice

The guarantee incremental = the same data loaded as one run, to the cent, on every node

Measured: 200 batches of 3 trades on one date; a restart equal too; batch time does not grow. Official numbers come only from full runs.

Six input contracts, JSONL on the wire, every check named

JSONL: one JSON object a line, UTF-8, .jsonl or .jsonl.gz; each line exactly its contract's fields. Key fields of the as-built contracts:

scenario-pv SPV · a trade × date × set	
TRADE_ID … TT_BOOK	the trade; its LE, desk, book
CURRENCY · BASE_PV	PVs' currency; value today
SCENARIO_PV [509]	per scenario; null = failed
RUN_KIND	FULL or INTRADAY
SCENARIO_SET_ID	and BATCH_ID: the run

pnl-vector PNLV · a trade × date × set	
VECTOR_ID · TRADE_ID	PV-<trade>-HS-509
PNL [509]	USD P&L, gain positive
PV · REPORTING_CCY	USD value today; USD
STATUS	ok, proxied or failed
RUN_KIND	as scenario-pv

fx-scenario FXS · a currency × date	
FX_SCENARIO_ID	FXS-<CCY>-HS-509
CURRENCY	AUD, CAD, EUR, JPY …
RATE	today, USD per one unit
RATES [509]	per scenario, the set's order
not sent?	derived from the FX history

scenario-set SSET · one row a date	
SCENARIO_SET_ID	HS-509
SCENARIO_DATES	509 days, oldest first
WINDOW_START … END	2024-09-17 … 2026-09-29
HORIZON_DAYS	1
REVALUATION	full or sensitivities

risk-hierarchy HIER · a node × date	
NODE_ID	MGMT~<id> or LEGAL~<id>
HIERARCHY · LEVEL	MGMT/LEGAL; bank…book
PARENT_ID · NAME	the tree; display name
BOOKS	every book beneath
LIMIT_VAR	1-day 99% limit, USD

portfolio PORT · a portfolio × date	
PORTFOLIO_ID · NAME	PF-…; scope PORT:<id>
TYPE	HIERARCHY, RULE or BASKET
DEFINITION	NODE, RULE or MEMBERS
OWNER	a user or a team
LIMIT_VAR	the portfolio's limit, USD

BUILT **Required on every record (2.9.4):** VALUE_TYPE, GRAIN with its id, CURRENCY, SOURCE_SYSTEM; GRAIN matches the id. Old files: `convert --set`

DRS-5201 length, null, NaN	**DRS-5206** a PV failed	**DRS-5207** no scenario FX	**DRS-5208** record fields wrong	**DRS-5209** against the pin	**DRS-5210** 2nd form or grain	**DRS-5202** trade twice	**DRS-5203** two runs

One explorer for any scope, and the same answers everywhere else



VAR DESK DESK-RATES IN EUR: the scope tree and tiles in EUR, the basket tray, the P&L path, and How this number was built naming the reporting currency, the input form, the scenario FX and the crosses via USD (2026-09-30).

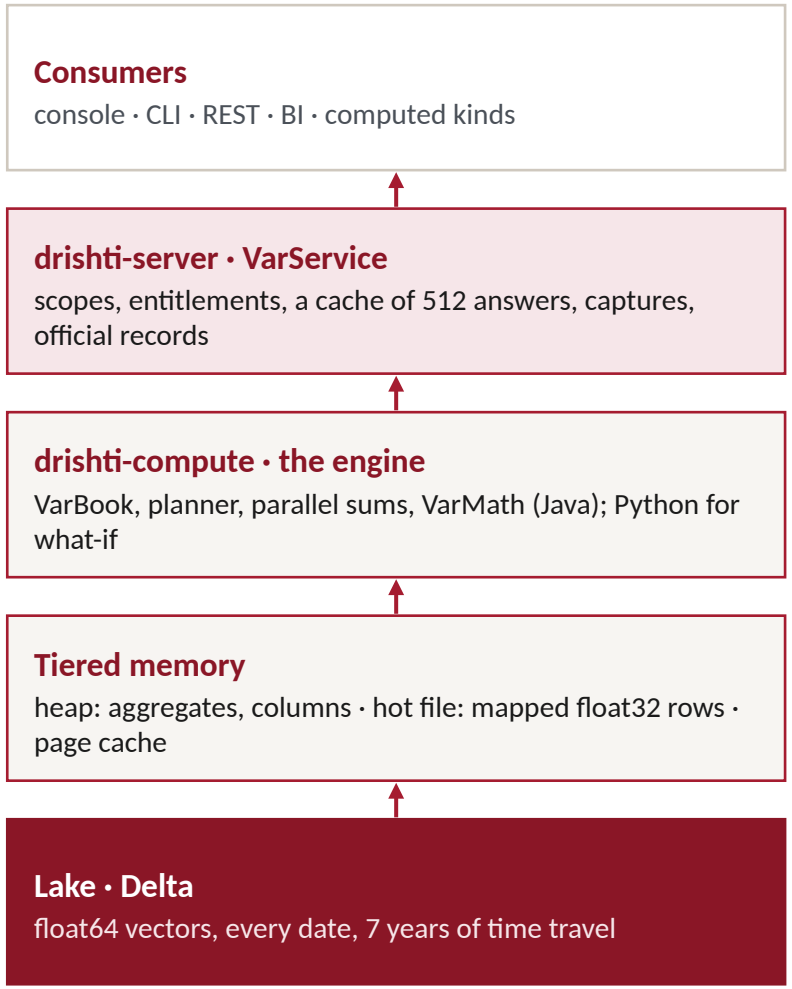
9–14 ms click to painted, a desk, answers fetched ahead (target 300 ms)

390 px on a phone the three columns stack, no sideways scroll

- **Scope tree and six tiles** MGMT, LEGAL or a configured tree (limit letters, 2.9.5) with VaR, ES and limit bars; VaR, ES, sum of standalone, diversification, limit use, backtest exceptions.
- **Five tabs** P&L path (509 days, worst 25, who lost on a day) · VaR/ES history and backtest · contributions · captured results · In other currencies.
- **Reporting currency picker** USD, EUR, CAD, JPY, GBP and More…; every panel follows; kept in the link; VAR … IN EUR.
- **How this number was built** the scope as resolved, its trades and positions, hidden and unknown ids, batch, data versions, method, tier, timing.
- **Baskets and pre-deal what-if** any mix of portfolios, positions, trades, counterparties and rules, live; a booked trade's vector × a factor through the Python engine.
- **Captures, official records** immutable captures, compare with now; each node's official history.
- **Everywhere else** trade (Trade currency), position, portfolio, desk and LE pages; drishti.py var … --ccy; /api/v1/var; VRES and VWIF; the BI VaR report; Home facts.
- **Governed** root name drishti.var.root-name; per-node positions or aggregate access; masks; every answer audited.

HOW IT SCALES

Why it is fast: vectors add, one date stays hot, sums run on every core



Vector math
509 doubles = 4,072 B a trade or position; hot rows float32 (2,036 B); sums in float64

Memory-mapped hot file
one file a date; only touched pages resident; 2 dates open, 60 kept on disk

Planner over aggregates
a node is its aggregate; Markets minus three trades = 4 vector operations

Parallel sums
slices on every core, added in a fixed order: the same bits every time

Currencies built ahead
five books in the background; 21.4 MB a currency a date on the demo

Per-trade-currency totals
a headline in any currency from a node's few currency sums, exact

Intraday increments
cost ∝ rows changed; Kafka: one commit and one apply per window

Cluster and history
Drishti*net replicas; VaR sharding designed; Delta history

Measured	Time	On
A node's VaR and ES	0.48 ms	tiered store, 100,000 trades
The whole bank from float32 rows	3.4 ms (≈ 60 GB/s)	tiered store, 100,000 trades
The whole bank from float64 rows	58 ms (≈ 70 GB/s)	heap tier, 1,000,000 trades
A basket of 50,000 scattered positions	2.6 ms + ≈ 45 ms ids	tiered store, 100,000 trades
First load of a date (after the lake read)	0.8–1.2 s	tiered store, 100,000 trades
Loading the demo date	572 ms	9,572 trades, the server

Bank scale, from first principles and the measured figures

Per business date	Arithmetic	Result	Basis
Vectors, float64 (the lake)	2.3M × 509 × 8 B	9.37 GB	arithmetic
Vectors, float32 (hot rows)	2.3M × 509 × 4 B	4.68 GB	arithmetic
Hot file, USD book	237.7 MB per 100k × 23	≈ 5.5 GB	*estimated*
Heap, one open date	≈ 80 MB per 100k × 23	≈ 1.8 GB	*estimated*
First load, after the lake read	0.8–1.2 s per 100k × 23	≈ 18–28 s	*estimated*
Reopen a date from its file	23 ms per 100k × 23	≈ 0.5 s	*estimated*
Whole bank from rows, resident	4.68 GB at ≈ 60 GB/s	≈ 80 ms	*estimated*
... cold, from NVMe	4.68 GB at ≈ 3 GB/s	≈ 1.6 s	*estimated*
A node, any size	its float64 aggregate	≈ 0.5 ms	**measured**
4 extra currencies, disk	250 MB × 4 per 100k × 23	≈ 23 GB a date	*estimated*
... 5 retained dates; build	23 GB × 5; 4.3 s per 100k × 23	≈ 115 GB; ≈ 99 s	*estimated*
USD hot files, 60 dates	5.5 GB × 60	≈ 330 GB disk	*estimated*
Lake, a year (Parquet ≈ 0.75×)	9.37 GB × 0.75 × 252	≈ 1.8 TB; ≈ 12 TB in 7 y	*estimated*

Position grain (2.9.4) keeps rows = positions: no trade matrix in the hot file. *Estimated = scaled linearly (× 23) from the measured 100,000-trade figures (VAR_GUIDE 27.3, 15.11); not measured at 2.3M.*

15,000 portfolios

- A portfolio is the sum of its positions: the cost is all portfolios' positions together × 509 additions.
- Assumed 130 positions each: 1.95M rows, 3.97 GB of float32 → ≈ 0.07 s at 60 GB/s, ≈ 0.8 s scattered; per currency, estimated.
- Roll-ups: a HIERARCHY portfolio is a node aggregate; whole books cost one add each; a node minus a few rows is aggregate − rows (built).
- Store them: 15k × 509 × 8 B ≈ 61 MB a date a currency; float32 halves it (proposed).

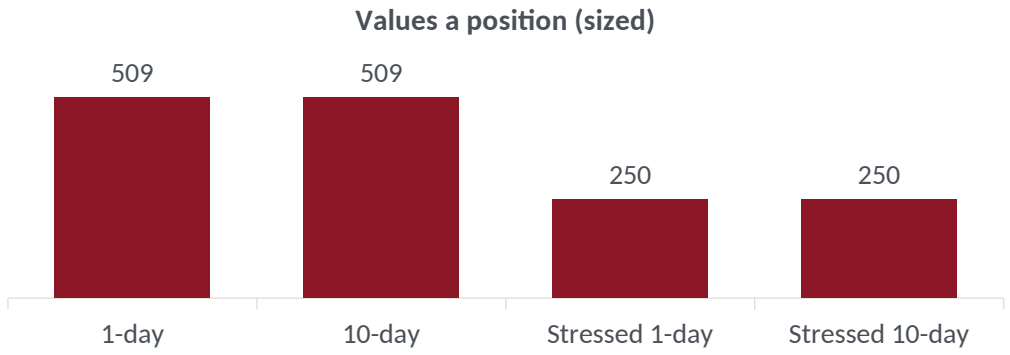
Totals and intraday

- Currency totals: 510 × 8 B per node per trade currency; (5,000 MGMT and LEGAL + ≈ 700 limit-letter nodes) × 10 ≈ 233 MB (assumed shape).
- Per changed trade or position: ≈ 11 node vectors (three trees) × 5 currency books ≈ 55 additions of 509: microseconds; the lake append dominates (≈ 1 s a batch).
- A headline in any currency: a node's few trade-currency sums, converted: microseconds.

10-day and stressed VaR: the added workload

2.9.6

Measure	Vector a position	× 1-day
1-day VaR and ES	509 scenarios (2 years)	1.00
10-day (b), overlapping	500 (509 − 9), sized 509	+ 1.00
Stressed 1-day VaR, ES	≈ 250 (12 months, e.g. 2008–09)	+ 0.49
Stressed 10-day	≈ 250 overlapping	+ 0.49
Total, sized	1,518 values a position	2.98



Per date and server	1-day only	All four	Label
Vectors, float64	9.37 GB	27.9 GB	ESTIMATED
Rows, float32	4.68 GB	14.0 GB	ESTIMATED
USD hot file	5.47 GB	16.3 GB	ESTIMATED
Heap, an open date	1.84 GB	2.23 GB	ESTIMATED
First open: lake read	18–70 s	54–209 s	ESTIMATED
First open: build	18–28 s	55–82 s	ESTIMATED
Other currency, asked	5.75 GB	+ 11.4 GB	ASSUMPTION
Local NVMe a server	458 GB	1,183 GB	ESTIMATED
RAM a server	76 GB	151 GB	ESTIMATED
Lake, 7 years	16.1 TB	48.2 TB	ESTIMATED

Method (ASSUMPTION): 10-day by (b), overlapping returns, the conservative case; (a) √10 × 1-day adds no data, × 1.49 only.

Currencies (ASSUMPTION): USD builds all four sets with the date; the others build 1-day ahead and the rest on first ask.

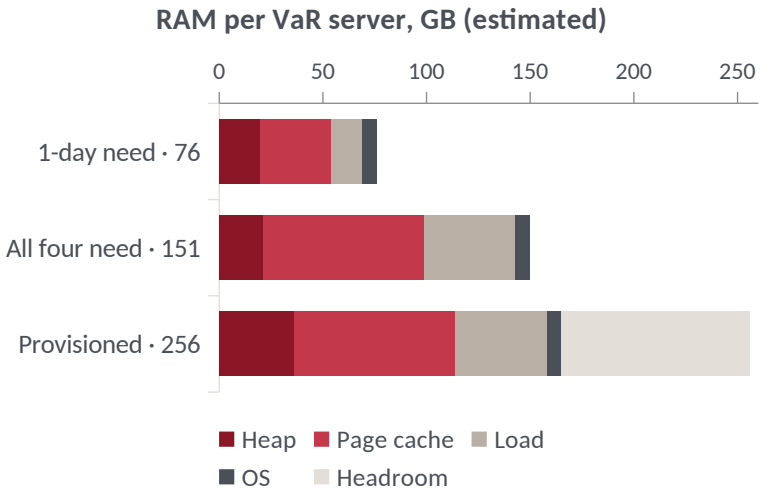
About 3× the 1-day footprint per date. With (a) √10 instead: × 1.49, vectors 18.5 TB over 7 years. Sums per measure are unchanged: a node 0.5 ms, the whole bank ≈ 80 ms for a 509-wide set, ≈ 40 ms for a stressed one (estimated).

Capacity: processes and memory, 1-day, 10-day and stressed

Processes, recommended one server holds each date's whole VaR

Role	No.	Holds and does	Status
Official EOD server	1	captures every node of every tree and measure; publishes (DRISHTI_VAR_OFFICIAL here only)	BUILT
Intraday writer	1	Kafka PV topic, one lake batch a 5-minute window, and files (DRISHTI_VAR_PV_STREAM here only)	BUILT
UI/API VaR replicas	2	each maps today and yesterday whole, applies intraday; Drishti*net failover	BUILT
Console hosts	2	4 workers each; load balancer, sticky sessions; one session secret	BUILT
Cluster, sharding	–	servers as one; a date's VaR split; consoles without stickiness	DESIGNED

Per VaR server	Need	Provisioned	Large (2×)
vCPU	32	64	96
-Xmx	21 GB	36g	60g
Memory budget, MB	14,300	23,000	46,000
RAM	151 GB	256 GB	512 GB
Local NVMe (tier.dir)	1.18 TB	3.8 TB	5.7 TB



CPU: 32 vCPU needed → 64

- Sums are bandwidth-bound: 24 threads gave ≈ 60 GB/s (MEASURED); all four sets of the bank ≈ 233 ms
- + 8 vCPU for a load, currency builds, intraday, requests; 32 ÷ 0.61 = 52 → 64

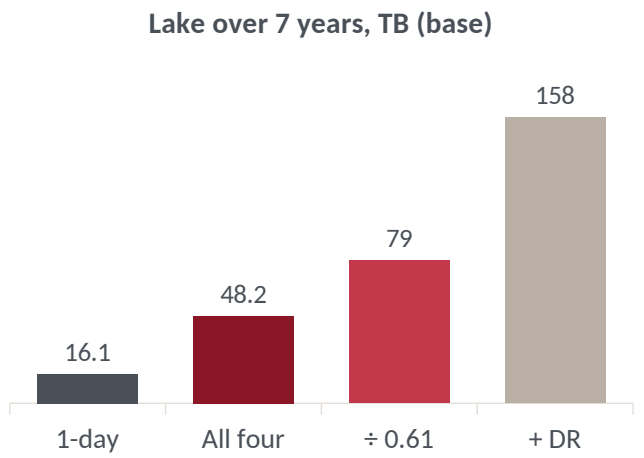
39% headroom: the projected peak uses at most 61% of what is provisioned: need ÷ 0.61 (× 1.64). Set DRISHTI_VAR_MEMORY_MB: the default ¼ of the heap (9 GB at 36g) refuses a 14 GB date.

Consoles: 2 hosts × 4 workers; 40 pages/s a worker (MEASURED); 50 users at once (ASSUMPTION): peak 25 pages/s ÷ 0.61 = 41 against one host's 160.

Capacity: storage over 7 years

Item	Arithmetic (all four sets)	1-day	All four	Label
P&L vectors, float64	27.9 GB × 0.75 × 1,764 dates	12.4 TB	37.0 TB	ESTIMATED
Row columns (terms)	2.3M × ≈ 100 B × 1,764 × 4 sets	0.4 TB	1.6 TB	ASSUMPTION
Scenario PVs as well	the vectors again, if both kept	+ 12.4	+ 37.0	ASSUMPTION
Trades, if held	1.5 KB × 2.3M × 1,764	6.1 TB	6.1 TB	ESTIMATED
Positions (membership)	41 B × 2.3M × 1,764	0.17 TB	0.17 TB	ESTIMATED
Official results	≈ 29 KB × 5,700 nodes × 1,764 × 2.98	0.29 TB	0.88 TB	ESTIMATED
Intraday batches	230k × 9.1 KB × 1,764 × 2	2.5 TB	7.4 TB	ASSUMPTION
Restatements	2% of dates × 20.9 GB × 1,764	0.25 TB	0.74 TB	ASSUMPTION
Delta log, checkpoints	daily 0.14 × 2.98 (often: 4.9)	0.14 TB	0.42 TB	ESTIMATED
Need, base	37.0 + 1.6 + 7.4 + 0.9 + 0.2 + 0.7 + 0.4	16.1 TB	48.2 TB	ESTIMATED
Provisioned (÷ 0.61)	39% headroom; DR doubles it	26.4 TB	79 TB	ESTIMATED
Need, upper	+ PVs, trades, frequent checkpoints	36.1 TB	95.7 TB	ESTIMATED

With a DR copy: 158 TB provisioned (base), 314 TB (upper). Delta or Iceberg, not both. 7 × 252 = 1,764 dates; Parquet ≈ 0.75 × raw; × 2.98 = all four sets.



Local NVMe, per server

- USD 16.3 GB × 60 dates 978 + currencies 161 + a load 44 = 1,183 GB ÷ 0.61 → 3.8 TB; rebuilt from the lake, never on EFS

The 250-day window

- 1-day and 10-day cut to 250: 1,518 → 1,000 values (× 0.66); float32 14.0 → 9.2 GB; NVMe ≈ 780 GB
- The lake shrinks (37.0 → 24.3 TB) only if the bank sends 250-day sets

Hardware plan with 39% headroom

1-day, 10-day and stressed VaR at 2.3M positions

provisioned = need ÷ 0.61

Item	No.	Need, each	Provisioned, each	Large (2×)
VaR servers (EOD, intraday, 2 UI)	4	one machine type	identical, any role	5 (+ history)
vCPU	–	32	64	96
RAM	–	151 GB	256 GB	512 GB
Heap (-Xmx)	–	21 GB	36g	60g
VaR memory budget	–	14,300 MB	23,000 MB	46,000 MB
Local NVMe (tier.dir)	–	1.18 TB	2 × 1.9 TB	4 × 1.425 TB
Console hosts	2	25 pages/s	4 vCPU, 8 GB, 4 workers	the same
Lake, 7 years (object store)	1	48.2 TB	79 TB; 158 with DR	158; 316 with DR
Shared state (EFS or NFS)	1	0.3 TB	0.5 TB	0.5 TB
Network, a date's first open	–	20.9 GB, ≤ 0.4 GB/s	≥ 10 Gbit/s	25 Gbit/s
Intraday traffic	–	17.5 MB a window	< 1 MB/s a server	< 2 MB/s
Kafka PV topic, 7 days	–	5.5 GB a day; 39 GB	64 GB a replica	128 GB

Does one server hold a date?

Yes. All four sets need 151 GB: one 256 GB server per role, 512 GB at 2×.

Sharding (DESIGNED) is needed beyond ≈ 5M positions (a date takes 4–10 min to open) or ≈ 10M (over 1 TB of RAM).

Assumptions to confirm

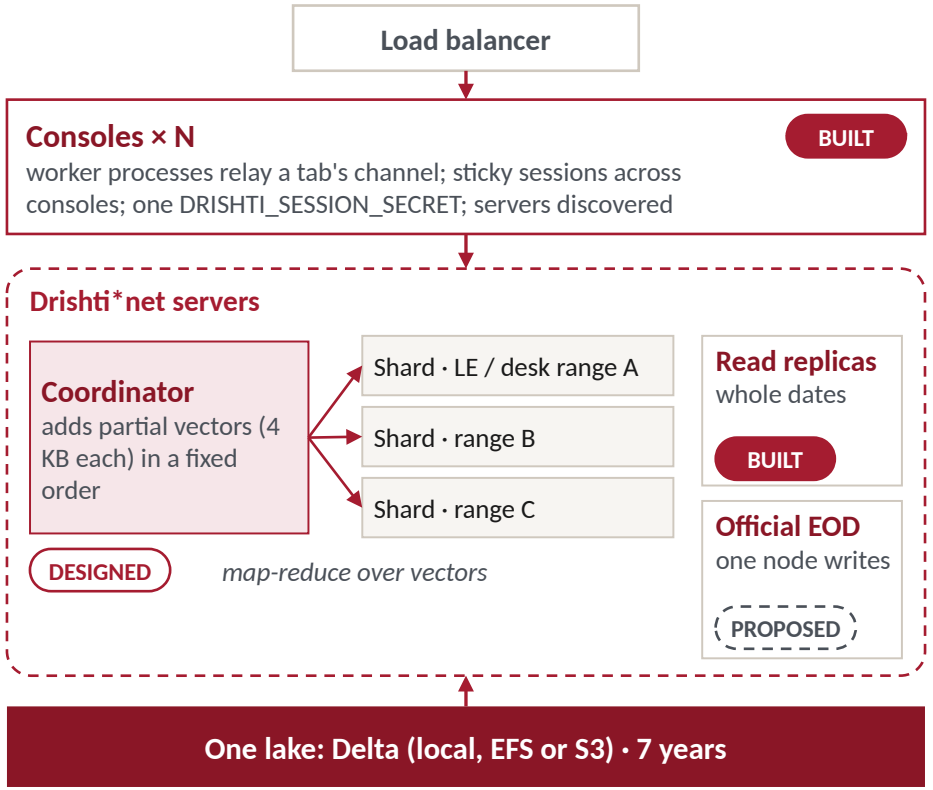
ASSUMPTION

- 50 of 200 users at once; peaks 5×
- 10-day by overlapping returns; stressed 1- and 10-day
- Intraday 230k rows a day, 120 windows
- 5,700 nodes; 2% restated; a DR copy
- Kafka 24 KB a record, kept 7 days

AWS example: m6id.16xlarge (64 vCPU, 256 GiB, 2 × 1.9 TB NVMe, 25 Gbit/s); large r6id.24xlarge; Fargate consoles; S3.
On premises: 2 × 16 cores (64 threads), 256 GB on every memory channel, 2 × 1.92 TB NVMe, 25 GbE; an S3-compatible object store.

Delta for history and reproducibility, Drishti*net for scale-out

Partition by business_date; 7 years of time travel full run replaces a date; restatement = new commit	BUILT
Vector column list<double>, float64 standard Parquet; a sum reads one column	BUILT
Compaction + checkpoints (lake maintain) many small intraday commits stay fast	BUILT
Sort by LE, desk, book; ZSTD + byte-stream split a node is a contiguous run of rows	DESIGNED
Z-order or liquid clustering by book and portfolio skip files for a desk's or portfolio's read	PROPOSED
Published node and portfolio vector table BI and peers read sums, not 2.3M rows	PROPOSED
Change data feed on the input tables follow intraday deltas downstream	PROPOSED
Delta inputs on S3 (reading) the Delta plugin reads s3:// tables	BUILT
Iceberg for VaR inputs the Iceberg plugin streams vectors too	2.9.5
Official publish to Iceberg and S3 today a local or EFS Delta root	2.9.5



To reach 2.3M positions and 15,000 portfolios, still to build

1 The 2.3M benchmark on a server, both formats (2.9.6)
2 VaR sharding across Drishti*net (scatter-gather)
3 The lake layout at scale: sort, ZSTD, file sizes
4 The 15,000 portfolio vectors built in a batch
5 Iceberg for VaR inputs (2.9.5)
6 Official results to Iceberg and S3 (2.9.5)
7 Several consoles without sticky sessions

Delta Lake or Iceberg for VaR: performance and recommendation

VaR answers are format-independent. Once a date is loaded, the engine computes from its memory-mapped hot files; the lake format matters only for loading, intraday commits, history and other engines.

≈ 0.5 ms
a node's VaR and ES · measured

≈ 80 ms
the whole bank at 2.3M positions · estimated

Where they differ	Delta Lake	Iceberg	Edge
First load of a date	replays its log from the last checkpoint	plans from manifests	about the same: both Parquet, Drishti's own readers
Intraday micro-batches (Kafka every 5 min, ≈ 120 commits a day)	a small log entry per commit, then checkpoints	rewrites its metadata file each commit; ≈ 0.2 GB by year 7 (estimated): needs snapshot and metadata management	**Delta**
Time travel and restatements	rebuilds a version from its log and checkpoints	jumps to a snapshot	**Iceberg, slightly**
Caches after another day's commit	equivalent proposed	built	**Iceberg**
Other engines (Spark, Athena, Trino, Snowflake, Databricks)	strongest with Spark and Databricks	broader multi-engine support	depends on the platform
Correctness	identical: the 2.9.5 equality tests prove Delta = Iceberg to the cent		same

Recommendation

Delta is the default (product decision), above all with Kafka intraday. Choose Iceberg where the data platform standardises on it, or for heavy time travel and multi-engine access. Either is a configuration switch.

2.9.5 Iceberg for VaR inputs; official publishing to Iceberg and S3; the equality tests

2.9.6 to be measured: the 2.3M benchmark on both formats

SCHEMA CHANGES

When the data's structure changes

A new field in the source data

No schema change. Each record's full JSON document is kept in the doc column, so a new field is readable at once; older rows simply lack it.

BUILT

A new promoted column

Iceberg: in-place schema evolution by field id: add, rename, reorder, widen.

Delta: in-place add of a nullable column, old files read as empty, optional backfill from the documents. Until then, adding one to an existing table rewrites the affected dates.

Connector files gain a pack's new columns on upgrade, with a backup.

BUILT Iceberg 2.9.6 Delta add

History is never rewritten

Time travel shows each version as it was; official records are append-only. Adding a field never invalidates the lake, and a figure of record can still be recomputed from its inputs.

BUILT

```
# before: the source sends A and B
{"id":"T1", "doc":{"A":1,"B":2}, "A":1}
# after: the source adds C; nothing to migrate, C is in doc at once
{"id":"T2", "doc":{"A":3,"B":4,"C":5}, "A":3}
# C promoted to a column: new rows carry it, old files read it as empty (null)
```

DEPLOYMENT OPTIONS

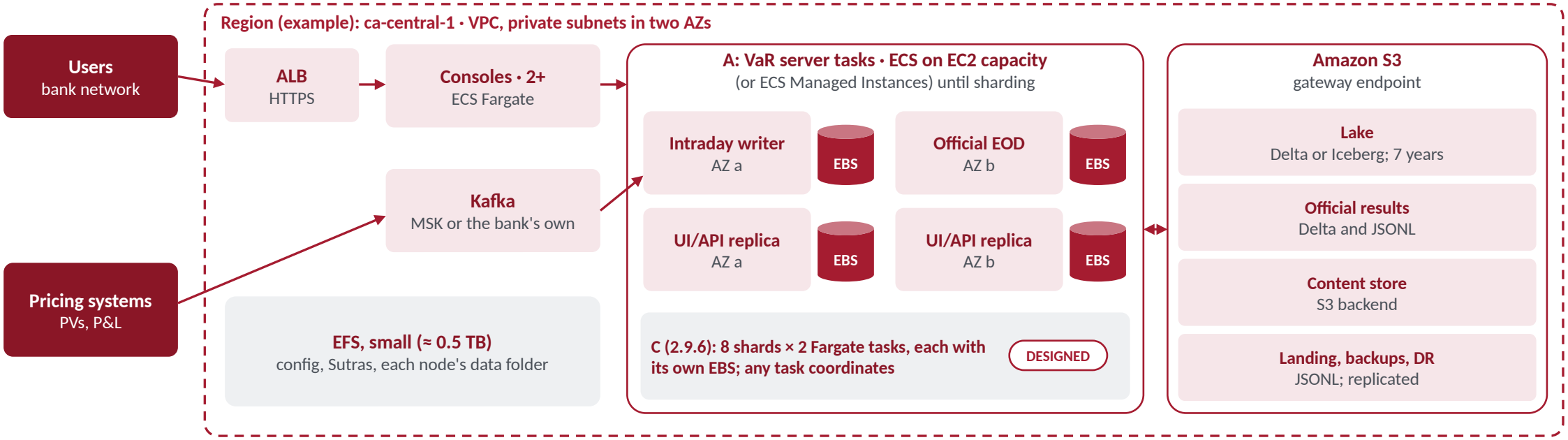
Delta or Iceberg, on premises or on AWS

	Delta Lake	Iceberg
On premises	BUILT VaR inputs from Delta tables	2.9.5 VaR and exposure inputs from Iceberg
	BUILT Official results: append-only Delta + JSONL	2.9.5 Official results to Iceberg
	BUILT Lake maintenance; 7-year floor for risk	BUILT Iceberg connector for other data
AWS	BUILT Reading Delta tables in S3	2.9.5 The AWS Glue catalog
	BUILT Official publish to an EFS Delta root	2.9.5 Iceberg AWS scripts
	2.9.5 Official publish to s3://	2.9.5 Iceberg admin, maintenance, caches

Above the storage plugins, one product packs, data contracts, connector names, engines, BI, UI and Drishti*net are the same on Delta and Iceberg. Hot files always on fast local disk.

See the companion decks: Drishti on AWS with Delta Lake, and Drishti on AWS with Iceberg.

Drishti on AWS: ECS, Fargate, S3 first, as little EFS as possible



VaR tier	Shape (full case: 1-day, 10-day, stressed)	Fits, with 39% headroom	Status
A · ECS on EC2 capacity	4 × m7i.16xlarge (64 vCPU, 256 GiB), 2 TB EBS gp3 each; the rest on Fargate	151 GB need → 256 GB machine	BUILT
B · √10 10-day on Fargate	4 Fargate tasks of 32 vCPU · 244 GB (the largest), 1.1 TB gp3	94 GB → 155 GB: memory yes; CPU to measure	2.9.6
C · sharded on Fargate	8 shards × 2 tasks of 8 vCPU · 44 GB, 250 GB gp3 each, one S3 lake	26 GB a task → 43 GB	DESIGNED · 2.9.6

Recommendation: Fargate for the consoles and lake tasks; A for the VaR servers until sharding; C, Fargate everywhere, once sharding ships (2.9.6). Hot files on an EBS gp3 volume per task: disposable, rebuilt from the lake. EFS only for what must be shared.

PROPOSED

AWS · EXAMPLE REGION CA-CENTRAL-1 · ON-DEMAND, USD

AWS cost estimate for a large bank

ASSUMPTION

Monthly, year 7, 39% headroom	1-day only	Full	Full, a year
VaR servers (4, EC2 capacity)	$9,124	$10,498	$125,976
EBS gp3 for hot files (4)	$898	$1,320	$15,840
Consoles (2, Fargate)	$317	$317	$3,805
Lake loads, maintenance (Fargate)	$31	$31	$369
S3: lake, results, content store	$258	$752	$9,021
DR copy (S3 replication)	$252	$753	$9,038
EFS and its backup	$190	$190	$2,280
Amazon MSK (3 brokers)	$531	$531	$6,375
Load balancer (ALB)	$31	$31	$371
NAT and interface endpoints	$173	$173	$2,074
Data transfer	$92	$92	$1,104
CloudWatch logs and metrics	$217	$217	$2,609
Total, on-demand	$12,119	**$14,910**	**$178,914**
with a 1-year Savings Plan	$9,596	$12,017	$144,201
with a 3-year Savings Plan	$7,443	$9,553	$114,637

Prices: AWS public on-demand list prices for the example Region, read 2026-10-27 (ASSUMPTION); quantities from the capacity slides (ESTIMATED). Verify before quoting.

The VaR tier: A, B or C

servers and EBS, $ a month

Option	On-demand	3-yr plan	+ Spot
A: EC2 capacity	$11,818	$6,618	–
B: √10 on Fargate	$8,633	$5,200	–
C: sharded, 24 × 7	$7,373	–	–
C: scale-in off-hours	$5,050	$2,966	**$2,785**

Honest on both sides

- One full copy costs more on Fargate: 8 tasks $3,640 against one EC2 server $2,954 a month
- C wins by needing 2 copies, not 4, scaling replicas in off-hours, Spot for second replicas, growing by adding tasks
- C beats A once A needs more than 2.5 full copies; A needs 4

Recommendation

A now, until sharding ships; C from 2.9.6: the cheapest ($2,785 against $6,618 a month with 3-year plans) and fully ECS-native.

From the full run to intraday, every business date

1 Full run lands

after the close: scenario set, every live trade, hierarchies, portfolios; JSONL into the lake, then the load signal

2 Date loads

every row checked; positions derived; nodes summed; the hot file built; currencies built ahead

3 Official record

every node of every tree captured from the full run, published append-only, with JSONL

4 Intraday

changed trades or positions only, files or 5-minute Kafka windows; answers say intraday; the record stays

Monitoring, as built	What it shows
GET /api/v1/var/status	loaded dates, trades, positions, nodes, load time, rejected, excluded, batch ids, refused dates
GET .../var/intraday/streams	topic, state, waiting trades, offsets, last ten windows, newest rejections
Provenance on every answer	inputs by system, rejections by code, intraday batches applied
Admin pages	Audit, Lakes (maintenance), the connector's Apply now
drishti.py var status	the same status from the command line

What to check each morning

- The newest date is loaded, from one run (one BATCH_ID)
- Refused rows and refused dates are zero, or explained
- The official record exists; no publishError
- Each system arrived in its expected form and grain
- Limit use: amber 75%, red 90%; the backtest light
- Intraday streams: nothing waiting long; no rejections

Not checked by the engine today: the batch's due time and its completeness against the trades booked (planned in the VaR guide); agree them with the pricing-system teams.

When something goes wrong, and the runbook

Situation	What the engine does	What the operator does
Bad rows (DRS-5201, 5206, 5207, 5208, 5209)	refuses each row, names it; never zero-fills	fix the rows, restate the date
A trade twice on a date (DRS-5202)	does not load the date	restate with one row per trade
Two runs or sets on a date (DRS-5203)	answers the date before, with a warning	restate as one run; signal
Second form or grain from a system (DRS-5210)	refuses: never a double count	fix the system's output
A date over the memory budget (DRS-5204)	503; never loads part of a date	raise memory-budget-mb or heap
Restatement	reloads; next result version; old kept	rewrite the whole date, new BATCH_ID
Publish fails (disk, root)	records publishError; capture stands	fix the root; capture again
Damaged or old hot file	reads the lake, rebuilds the file	nothing
Restart, or a Kafka replay	rebuilds from the lake; writes nothing twice	nothing

Runbook 1 Check status and provenance → 2 Name the code → 3 Fix at source → 4 Restate the whole date → 5 Signal; verify the version

The engine never guesses: a bad row is refused and named; a date that cannot load as one run is not loaded, and the day before is answered with a warning rather than a wrong number.

Every number reproducible, every day backtested

Reproducibility, as built

Versions on every answer
inputs' data versions, batch ids, scenario set, method, engine version

Digests
the position rules' text and digest, the membership hash

Append-only official records
a restatement adds a result version; the earlier is kept

7-year retention
vectors with time travel, captures and records; maintenance never goes below

Recomputable
the same vectors and engine version give the same number years later

Exact arithmetic
a node's figures equal the lake's to the bit; components add up to VaR

Backtesting at 99% losses bigger than VaR are breaches

Green 0–4	Amber 5–9	Red 10+

Kupiec test too many or too few breaches?

Christoffersen test do breaches cluster?

Official mode a node's official history: each day's hypothetical P&L against the day before's VaR

Replay today's members over past scenario days: in-sample, labelled not a regulatory backtest

Demo, replayed FX desk: 3 breaches in 249 days, green; the bank: 5, amber

Short histories limits scale with the days; not a regulatory zone before 250

Not built, to disclose age-weighted and filtered methods, ES backtests; 10-day and stressed VaR ship in 2.9.6. The hypothetical P&L is the engine's own; a bank's own hypothetical P&L table replacing it is planned.

Kupiec test (Paul Kupiec, 1995): is the number of VaR breaches (losses bigger than VaR) consistent with 99%, about 2–3 in 250 days?
Christoffersen test (Peter Christoffersen, 1998): do breaches cluster together, a sign the model reacts too slowly when markets turn?
Basel traffic light: breaches in 250 days at 99%: 0–4 green, 5–9 amber, 10+ red.

Independent checks, entitlements, audit, change control

Independent recomputation

PROPOSED

- A checker recomputes the VaR and ES of the bank and every desk from the files sent, and compares with the engine
- Exits non-zero on any difference: fit for a daily job
- The official record names its inputs' versions and batch ids: a recomputation reads exactly that data

Entitlements

- Per role and node: positions, or aggregate only (drishti.var.access)
- A scope is computed over the trades you may see; the rest counted
- Masks apply; answers cached per view, never shared across rights
- Saving baskets and captures needs the token scope risk:write

Audit

- Every headline, capture, basket change, reload and official run (Admin → Audit)
- A headline's entry: date, data version, view, VaR, ES, trades, hidden, members' digest, time
- Reloads and official captures: administrators only; official numbers kept 7 years

Change control of settings

PROPOSED

- Window, confidence, rank rule, currencies, pins: in application.yaml, shown by GET /status
- Every answer is labelled with its confidence and window; per-request windows are never official
- Pins read at start: apply from the next date loaded; provenance records them

PROPOSED marks the practice: a daily independent job, and comparing a setting change per request before making it; the mechanisms are built.

Numbers of record you can reproduce, and what comes next

Official, append-only

Every MGMT and LEGAL node, every date, from the morning run; a restatement adds a result version. Published to risk-results/var-official (Delta, appendOnly) with JSONL beside it.

Reproducible

Each answer names its inputs' data versions, batch ids, scenario set, position rules' digest, membership hash and engine version; a capture keeps the vector itself.

Kept 7 years

Vectors and their time travel, official captures and records; lake maintenance never vacuums the risk domains below 7 years.

Entitled and audited

Per role and node: positions or aggregate; masks on rows; every headline, capture, basket, reload and official run audited.

New in 2.9.4, built

- Any input contract: PVs or P&L, by trade or position
- Any reporting currency; five built ahead
- Intraday from files and Kafka micro-batches
- Window, confidence and root name as settings
- JSONL everywhere; the PV scripts (tools/pvgen)

Planned

- 2.9.5: Iceberg for VaR inputs and official records; hierarchies by columns
- 2.9.6: 10-day and stressed VaR; the 2.3M benchmark; Delta column add
- Sharding across Drishti*net (designed); the lake layout at scale
- What-if pricing through QLRisk, once its API is stable
- Age-weighted and filtered methods; ES backtests; a bank's own hypothetical P&L
- Cubes, overnight precomputation; the common risk lake

Key insight Official numbers come only from full runs, and each can be recomputed from its inputs years later.

A phased rollout with a parallel run

0 Prepare

- Environment: Delta, on premises or AWS
- Settings and entitlements agreed
- The 2.3M benchmark on the target server

1 Pilot desk

- The desk's systems onboarded
- Daily full runs and official records
- Parallel run against the existing VaR

2 A legal entity

- All its desks and systems
- LEGAL tree and limits
- Intraday switched on

3 The bank

- Every system, every tree
- ≈ 15,000 portfolios; ≈ 700 limit letters
- BI reads the official results

4 Cutover

- Sign-off
- Drishti's records become the numbers of record
- The existing VaR retired

Parallel run and reconciliation

- Each node, each date: the existing VaR beside Drishti's official record (the var-official table)
- Explain every difference by its cause: inputs (refused, excluded, unknown ids), method (window, confidence, rank rule), FX (scenario FX, not today's rate), membership (overlaps counted once), model
- Track the backtest light on the official history as it builds up

Sign-off criteria

PROPOSED

- Agreed tolerance per node; every difference explained
- No unexplained refusals over an agreed run of days
- Model validation of the method settings and the controls
- Runbook rehearsed: a restatement, a failed publish, a restart
- Backtest light not red; entitlements and audit reviewed

A proposed plan: generic, no dates. Each phase uses built capabilities; limit letters need the 2.9.5 hierarchies, the 10-day and stressed measures 2.9.6.

GLOSSARY

The terms used in this deck

Term	Meaning
VaR	the loss not exceeded at the confidence: 99% = minus the 6th worst of 509
ES	expected shortfall: minus the mean of the 13 worst P&Ls at 97.5%
Scenario	a historical day's market move, applied to today's trades
Scenario set	the scenario days of a date (HS-509), oldest first
Scenario PV	a trade's or position's value under one scenario, in its own currency
Path P&L	the P&L vector: one P&L per scenario (509 numbers)
Form · grain	PV or P&L · one record a trade, or one a position
Scope	any set of trades or positions: bank, LE, desk, book, portfolio, rule, basket
Node	a point of a hierarchy; its vector precomputed
Limit letter	a node of a hierarchy configured by columns, with a VaR limit
Position	trades grouped by the pack's rules, or sent whole
Portfolio	a stored scope: HIERARCHY, RULE or BASKET

Term	Meaning
Component VaR	a part's share of VaR (Euler); parts add up to it
Incremental VaR	VaR(scope) − VaR(scope without the part)
Marginal VaR	component VaR per million of size
Diversification	the sum of standalone VaRs minus the VaR
Scenario FX	each scenario's exchange rates, used to convert
Reporting currency	the currency an answer is in; five built ahead
Hot file	a date's vectors, memory-mapped from local disk
Official record	a node's end-of-day VaR and ES, append-only
Result version	1, 2, 3 … as a date is restated
Provenance	how a number was built: inputs, versions, refusals
Breach (exception)	a day whose loss exceeds the VaR tested against
Headroom 39%	provisioned = need ÷ 0.61

Thank you

Any data. Any domain. One grammar.

One engine, every input: PVs or P&L, by trade or by position, any currency, files or Kafka.

Drishti • Ashutosh Sinha • October 2026 • VaR engine 2.9.4

Copyright © 2026 Ashutosh Sinha. All rights reserved. Proprietary and confidential.